

Tracking the DNA of Your Android Code



In my previous article ‘Do You Know the Tricks to Track Your Code’s DNA?’, we explored a process that developer teams can follow to manage the source code for software-related product development, using the Black Duck Suite. In this article, we will explore Android-development-related complexities, and the need to track code and the related licences.

Consider this: the Android-based Samsung Vibrant (SGH-t959) phone has a section containing legal acknowledgements of all the open source code on the phone—and it is over 8,000 lines long. It specifically acknowledges hundreds of copyright holders. For many, this acknowledgement is specific to individual files, or to a list of files. To comply with its publishing obligations, Samsung provides a download of the files on its Open Source Release Centre website, where anyone can access the files. Any files that do not comply with file-level obligations (such as removing copyright statements, incorrectly adding copyright statements, not documenting modifications and others), could be relatively apparent to copyright holders and/or others.

For many contributors to the open source community, this acknowledgement is all they ask, in return for the use of their software. Companies should, and easily can, put together tools and processes to make sure this acknowledgement happens. The best practice is to carefully stipulate which licences, components, copyrights and files are in their code, and what obligations result from that mix of third-party software. But this task is easier said than done.

In the previous article (<http://www.linuxforu.com/news/do-you-know-the-trick-to-track-your-codes-dna/>), we had explored how managing these complex source-code reviews can become a straightforward process with tools such as Black Duck software.

Now, since the subject of this article is Android development, let’s first focus on the licensing complexities, and the issues that necessitate tracking the source code and addressing the related obligations. It would then be worthwhile to evaluate how, with a tool like Black Duck’s Android Fast Start, a bundled software and service, you can manage the continual changes and associated operational, legal, and security challenges in a seamless process integrated into your Android development environment.

Android: The complexity within

The Android operating system for mobile devices was made available as open source software under an Apache licence in 2008. Android contains internal components (the platform) and external components (the Linux kernel and WebKit, under the GPL and LGPL licences, apart from various other components or projects, copyrighted by different owners). While Android is feature-rich and free in terms of acquisition costs, it’s not immune to obligations, and requires proper management and control, says Rohit Sharma, vice president, Lyra Infosystems (P) Limited, the official representative partner of Black Duck Software Inc, in the Indian sub-continent.

Let’s explore this technical complexity at length, looking at it from the perspective of developers—third-party companies that develop software components for device manufacturers and OEMs. The Android open source project is sponsored by Google, created using the GPLv2-licensed Linux kernel, and represents a fork in the Linux kernel. Despite its roots in the GPL, Android’s collection of 185 different sub-components is written under 19 different open source licences—most reciprocal, and not all OSI-approved. While the majority of Android code contributed by Google is governed by the Apache 2.0 licence, a number of components that mobile developers rely on are governed by other licences, explains Sharma.

“Android’s open source software assets are grouped into external and internal categories. Android’s two major external components—the Linux kernel and WebKit—are governed by reciprocal licences (GPLv2 and LGPL). In addition to the two major external components, an additional 30+ internal components (*dbus*, *grub*, *emma*, *e2fsprogs*, *bluez*, *Bison*, etc) also use reciprocal licences (GPL, LGPL, CPL, etc). Twenty-eight components use the GPL and five use the LGPL, while others use non-OSI licences such as the OpenSSL combined licence and the Bzip2 licence,” he adds.